

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

SESSION 5

Dynamic parallelism and asynchronous operation

Groothuis Thomas (r0955783)
De Smet Feniks (r0891378)

Academic year 2025–2026

Contents

1 Problem & Goals **2**

2 Methodology **2**
2.1 CPU Version 2
2.2 Synchronized Version 2
2.3 Asynchronized Version 2
2.4 Dispatch Kernel 3

3 Results **3**

4 Conclusion **4**

A Code Implementations **5**
A.1 CUDA code 5
A.2 Data Collection Script 7

1 Problem & Goals

This lab reviews the concepts of synchronous and asynchronous operations. When processing data, we can choose between two approaches: synchronous or asynchronous execution. In a synchronous approach the CPU prepares the work, passes it to the GPU and waits for the result. In an asynchronous approach the CPU offloads the work to the GPU and immediately continues with other tasks. Both approaches are necessary in certain circumstances: asynchronous execution introduces the challenge of more complex control, while synchronous execution is simply slower. The goal of this lab is to test these two modes of operation and evaluate their benefits and drawbacks. We do this by reusing our reduction kernel from Lab Session 2, extending it to perform a summation, calculate a product, and find the minimum and maximum elements. This is done across four independent arrays. Finally, we execute this workload both synchronously and asynchronously to compare the performance results.

2 Methodology

The program is designed to accept a single command-line argument N, which defines the array size. The execution configuration is set to 1024 threads per block, and the maximum number of blocks required is calculated as (N + threadsPerBlock - 1) / threadsPerBlock. For memory management, we allocate host memory for four input arrays of size N and one output array of four floats. On the device side, we allocate corresponding memory for four input arrays of size N alongside four output arrays of size maxBlocks. Additionally, we initialize four distinct CUDA streams.

These components serve as the foundation for evaluating our CPU, synchronous GPU, and asynchronous GPU implementations. Each approach is encapsulated in a dedicated function that is responsible for executing the workload and returning its total execution time. To accurately capture this duration, every function utilizes the std::chrono library. A start timestamp is recorded at the very beginning of the function, and an end timestamp is recorded just before returning the calculated time in milliseconds.

2.1 CPU Version

In the CPU baseline version, we rely on standard for loops to process the data. Each operation is applied to its designated input array. For the summation and product, we iterate through all elements to calculate the total sum and the total product. For the minimum and maximum search, we initialize the extremum with the first element of the array and then perform a linear comparison throughout the rest of the list. All of these operations are executed strictly sequentially, and each final result is stored at the correct index in the output array.

2.2 Synchronized Version

In the synchronized version, we execute a loop for each of the four operations. First, the CPU generates an input array for the current operation and copies it to the GPU using a synchronous memory transfer. Next, we perform the reduction process on this data. This step uses the same reduction logic as developed in Lab 2, but this time the kernel launch is handled by a dedicated dispatchKernel function, which will be detailed in a later subsection. Once the reduction for the current operation is complete, we copy the final result into the corresponding index of the host's output array and move on to the next operation. Finally, to ensure all tasks are fully completed, we explicitly synchronize the device one last time before stopping the timer.

2.3 Asynchronized Version

The asynchronized version requires one additional parameter: the array of four initialized CUDA streams. Its structure closely mirrors the synchronized version, with the critical distinction that all memory transfers (both host-to-device and device-to-host) are executed asynchronously via cudaMemcpyAsync. These copies, alongside the kernel launches via dispatchKernel, are explicitly queued in the CUDA stream corresponding to the current operation. This design establishes a pipelined workflow: for each iteration, the host generates the input data and subsequently offloads the memory transfers and computational tasks to the respective stream. Because the CPU does not stall, it can immediately proceed to generate data for the next operation while the GPU processes the active streams. This also means that explicit synchronization via cudaDeviceSynchronize() is no longer needed within the reduction loop. Since commands issued to the same stream are guaranteed to

execute in sequence, the GPU automatically ensures that the next step only begins once the previous one has completed.

2.4 Dispatch Kernel

The dispatchKernel function requires several arguments: the operation identifier (op), the grid and block dimensions (blocks and threads), pointers to the device input and output arrays, the number of elements to process, and finally, the CUDA stream. By default, this stream parameter is set to 0. This allows the same function to be seamlessly reused in the synchronous implementation, where stream 0 effectively represents the default, synchronous CUDA stream. The operation identifier is then used to evaluate which specific blockReduction kernel to invoke. The kernel is launched using the execution configuration $<<< blocks, threads, 0, stream >>>$ (where 0 stands for the dynamically allocated shared memory size), followed by the standard kernel arguments: the input array, the output array, and the number of elements.

The blockReduction kernels are implemented using C++ templates. Rather than a standard type, this template uses an integer parameter to represent the specific mathematical operations. It largely follows the reduction logic from Lab 2, but the actual computation is handled by a device function called applyOp. This function takes two arguments: the element at index i and its corresponding neighbor within the input array. Because the applyOp device function is also templated based on the operation integer, it correctly applies the required mathematical operation to both arguments at compile-time.

3 Results

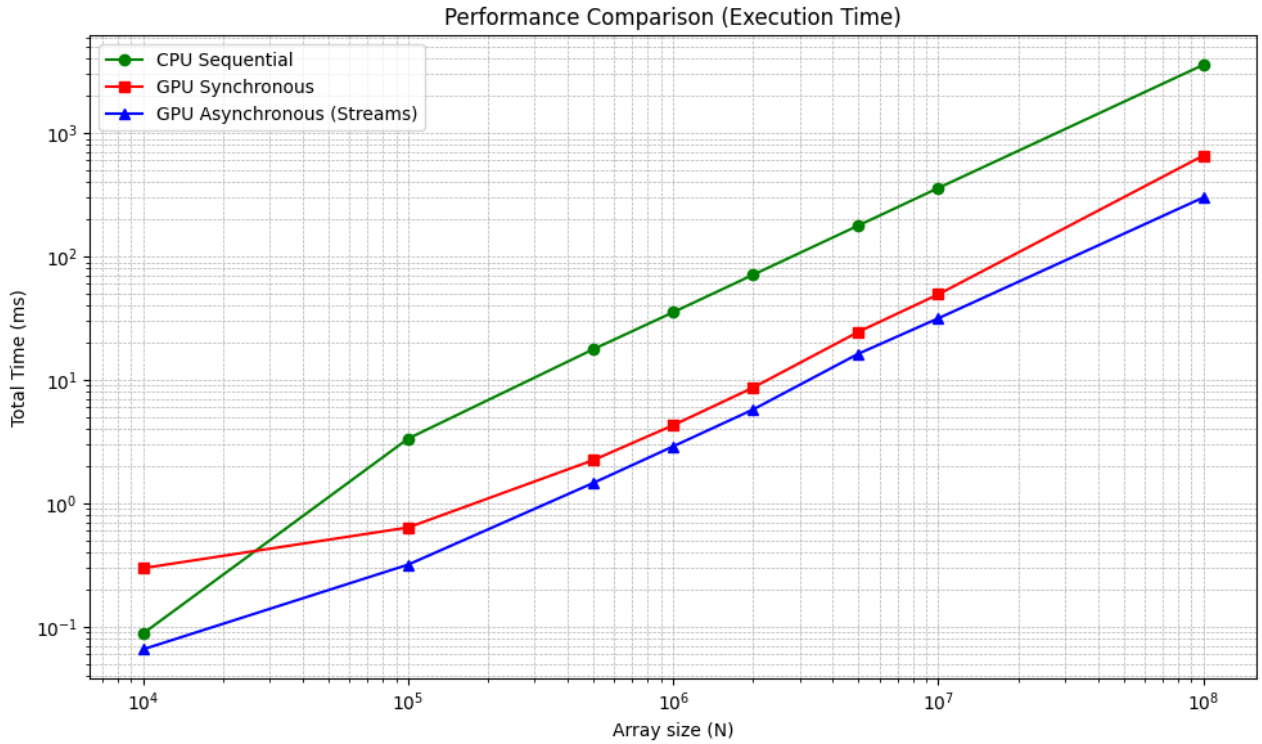


Figure 1: Execution Time

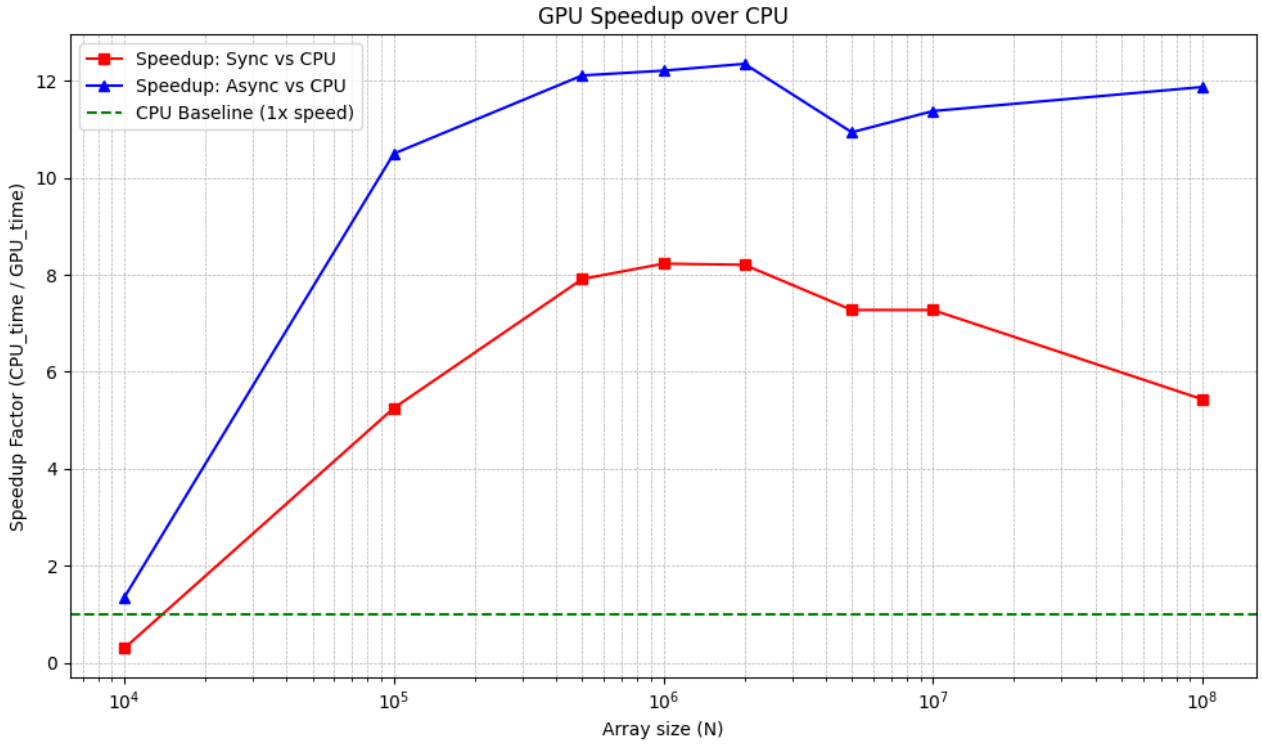


Figure 2: Speedup

Figure 1 demonstrates that the asynchronous GPU implementation outperforms the synchronous version, and both significantly surpass the sequential CPU baseline for larger datasets. For very small array sizes, the CPU is initially faster due to the overhead of PCIe memory transfers. However, as the workload increases, the GPU's parallel processing power dominates. As shown in 2, the asynchronous version achieves a peak speedup factor of approximately 12x compared to the CPU baseline, whereas the synchronous version peaks at around 8x. The asynchronous approach is significantly faster because it utilizes CUDA streams to overlap the memory transfers of the next operation with the kernel execution of the current one. Furthermore, this non-blocking behavior allows the CPU to concurrently generate new input arrays without stalling. In 2, a noticeable dip in the speedup factor occurs around $N = 5 \cdot 10^6$. This is caused by the arrays exceeding the capacity of the fast internal cache memory (L3 on the CPU and L2 on the GPU), which forces the hardware to rely on slower main memory and highlights the PCIe transfer bottleneck. As the array sizes grow even further, the synchronous speedup steadily declines because its execution becomes severely bottlenecked by these sequential memory transfers. The asynchronous version, however, successfully hides this transfer latency through its pipelined overlap, allowing its speedup to recover and plateau near its maximum potential.

4 Conclusion

In conclusion, the results demonstrate that the viability of GPU acceleration heavily depends on both the workload size and the execution strategy. For small datasets, the overhead of PCIe memory transfers outweighs the computational benefits, making a sequential CPU execution more efficient. Furthermore, the synchronous GPU implementation reveals significant bottlenecks for memory-bound workloads. Because the hardware remains idle during strictly sequential memory transfers and kernel executions, the overall efficiency degrades rapidly as the dataset size increases.

Asynchronous execution effectively mitigates these bottlenecks. By leveraging CUDA streams, the system achieves a pipelined workflow: the GPU can concurrently perform memory transfers for one operation while executing mathematical calculations for another. This parallelization successfully hides the latency of the data transfers, yielding a substantially higher speedup. However, the results also illustrate that even highly optimized asynchronous implementations are ultimately constrained by physical hardware limits. Once the

maximum PCIe bandwidth and memory capacities are saturated, the performance speedup plateaus, defining the absolute upper bound of the system's capabilities.

A Code Implementations

A.1 CUDA code

```cpp
#include <iostream>
#include <cuda_runtime.h>
#include <random>
#include <chrono>
#include <algorithm>

#define OP_SUM 0
#define OP_PROD 1
#define OP_MIN 2
#define OP_MAX 3

template <int OP>
__device__ inline float applyOp(float a, float b) {
    if (OP == OP_SUM) return a + b;
    if (OP == OP_PROD) return a * b;
    if (OP == OP_MIN) return fminf(a, b);
    if (OP == OP_MAX) return fmaxf(a, b);
    return 0.0f;
}

template <int OP>
__global__ void blockReduction(float *d_in, float *d_out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if ((threadIdx.x % (2 * stride)) == 0) {
            int neighbour = i + stride;
            if (neighbour < n) {
                d_in[i] = applyOp<OP>(d_in[i], d_in[neighbour]);
            }
        }
        __syncthreads();
    }

    if (threadIdx.x == 0 && i < n) {
        d_out[blockIdx.x] = d_in[i];
    }
}


void generateArray(float* a, int N, int op) {
    for (int i = 0; i < N; i++) a[i] = 0.99f;
}

void dispatchKernel(int op, int blocks, int threads, float* d_in, float* d_out, int
    numElements, cudaStream_t stream = 0) {
    if (op == OP_SUM) blockReduction<OP_SUM><<<blocks, threads, 0, stream>>>(d_in, d_out,
    numElements);
    else if (op == OP_PROD) blockReduction<OP_PROD><<<blocks, threads, 0, stream>>>(d_in,
    d_out, numElements);
    else if (op == OP_MIN) blockReduction<OP_MIN><<<blocks, threads, 0, stream>>>(d_in, d_out
    , numElements);
    else if (op == OP_MAX) blockReduction<OP_MAX><<<blocks, threads, 0, stream>>>(d_in, d_out
    , numElements);
}

float runCPU(int N, float* h_data[4], float* h_result) {
    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 4; i++) {
        generateArray(h_data[i], N, i);
        float acc = h_data[i][0];
        if (i == OP_SUM) {
            acc = 0;
            for(int j=0; j<N; j++) acc += h_data[i][j];
        } else if (i == OP_PROD) {
```

```
            acc = 1;
            for(int j=0; j<N; j++) acc *= h_data[i][j];
        } else if (i == OP_MIN) {
            for(int j=1; j<N; j++) acc = std::min(acc, h_data[i][j]);
        } else if (i == OP_MAX) {
            for(int j=1; j<N; j++) acc = std::max(acc, h_data[i][j]);
        }
        h_result[i] = acc;
    }
    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<float, std::milli>(end - start).count();
}

float runSync(int N, int threadsPerBlock, float* h_data[4], float* d_data[4], float* d_out
    [4], float* h_result) {
    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 4; i++) {
        generateArray(h_data[i], N, i);
        cudaMemcpy(d_data[i], h_data[i], N * sizeof(float), cudaMemcpyHostToDevice);

        int numElements = N;
        float *d_in_tmp = d_data[i];
        float *d_out_tmp = d_out[i];

        while (numElements > 1) {
            int blocks = (numElements + threadsPerBlock - 1) / threadsPerBlock;
            dispatchKernel(i, blocks, threadsPerBlock, d_in_tmp, d_out_tmp, numElements);
            cudaDeviceSynchronize();
            numElements = blocks;
            std::swap(d_in_tmp, d_out_tmp);
        }
        cudaMemcpy(&h_result[i], d_in_tmp, sizeof(float), cudaMemcpyDeviceToHost);
    }
    cudaDeviceSynchronize();
    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<float, std::milli>(end - start).count();
}

float runAsync(int N, int threadsPerBlock, float* h_data[4], float* d_data[4], float* d_out
    [4], float* h_result, cudaStream_t streams[4]) {
    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 4; i++) {
        generateArray(h_data[i], N, i);
        cudaMemcpyAsync(d_data[i], h_data[i], N * sizeof(float), cudaMemcpyHostToDevice,
    streams[i]);

        int numElements = N;
        float *d_in_tmp = d_data[i];
        float *d_out_tmp = d_out[i];

        while (numElements > 1) {
            int blocks = (numElements + threadsPerBlock - 1) / threadsPerBlock;
            dispatchKernel(i, blocks, threadsPerBlock, d_in_tmp, d_out_tmp, numElements,
    streams[i]);
            numElements = blocks;
            std::swap(d_in_tmp, d_out_tmp);
        }
        cudaMemcpyAsync(&h_result[i], d_in_tmp, sizeof(float), cudaMemcpyDeviceToHost,
    streams[i]);
    }
    cudaDeviceSynchronize();
    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<float, std::milli>(end - start).count();
}

int main(int argc, char* argv[]) {
    if (argc != 2) return 1;

    int N = atoi(argv[1]);
    int threadsPerBlock = 1024;
    int maxBlocks = (N + threadsPerBlock - 1) / threadsPerBlock;

    float *h_data[4];
```

```
    float *h_result;
    cudaMallocHost(&h_result, 4 * sizeof(float));

    float *d_data[4], *d_out[4];
    cudaStream_t streams[4];

    for (int i = 0; i < 4; i++) {
        cudaMallocHost(&h_data[i], N * sizeof(float));
        cudaMalloc(&d_data[i], N * sizeof(float));
        cudaMalloc(&d_out[i], maxBlocks * sizeof(float));
        cudaStreamCreate(&streams[i]);
    }

    float cpuTime = runCPU(N, h_data, h_result);
    float syncTime = runSync(N, threadsPerBlock, h_data, d_data, d_out, h_result);
    float asyncTime = runAsync(N, threadsPerBlock, h_data, d_data, d_out, h_result, streams);

    std::cout << N << " " << cpuTime << " " << syncTime << " " << asyncTime << std::endl;

    for (int i = 0; i < 4; i++) {
        cudaFreeHost(h_data[i]);
        cudaFree(d_data[i]);
        cudaFree(d_out[i]);
        cudaStreamDestroy(streams[i]);
    }
    cudaFreeHost(h_result);

    return 0;
}
```

A.2 Data Collection Script

```
import subprocess
import csv
import matplotlib.pyplot as plt

# Compileer de code
print("Compiling session_5.cu...")
subprocess.run(["nvcc", "-O3", "session_5.cu", "-o", "session_5"], check=True)

# Configuratie
sizes = [10_000, 100_000, 500_000, 1_000_000, 2_000_000, 5_000_000, 10_000_000, 100_000_000]
runs_per_size = 20
csv_file = "results_session5.csv"

# Data containers
cpu_times = []
sync_times = []
async_times = []

for N in sizes:
    print(f"Running N = {N}")
    cpu_tot, sync_tot, async_tot = 0, 0, 0

    for _ in range(runs_per_size):
        result = subprocess.run(["./session_5", str(N)], capture_output=True, text=True)
        parts = result.stdout.strip().split()

        if len(parts) < 4:
            print(f"Warning: unexpected output: {result.stdout}")
            continue

        cpu_tot += float(parts[1])
        sync_tot += float(parts[2])
        async_tot += float(parts[3])

    cpu_times.append(cpu_tot / runs_per_size)
    sync_times.append(sync_tot / runs_per_size)
    async_times.append(async_tot / runs_per_size)

    print(f"N={N} | CPU={cpu_times[-1]:.2f} ms | Sync={sync_times[-1]:.2f} ms | Async={
    async_times[-1]:.2f} ms")

```

```
# Opslaan in CSV
with open(csv_file, mode='w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(["N", "CPU_ms", "Sync_ms", "Async_ms"])
    for i, N in enumerate(sizes):
        writer.writerow([N, cpu_times[i], sync_times[i], async_times[i]])

# Plotten
plt.figure(figsize=(10,6))
plt.plot(sizes, cpu_times, marker='o', color='green', label="CPU Sequential")
plt.plot(sizes, sync_times, marker='s', color='red', label="GPU Synchronous")
plt.plot(sizes, async_times, marker='^', color='blue', label="GPU Asynchronous (Streams)")

plt.xlabel("Array size (N)")
plt.ylabel("Total Time (Generation + Computation) (ms)")
plt.title("Performance Comparison: CPU vs GPU Sync vs GPU Async")
plt.xscale("log")
plt.yscale("log")
plt.grid(True, which="both", ls="--", lw=0.5)
plt.legend()
plt.tight_layout()
plt.show()
```